Labo 11 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Deux VM Compose contre un cluster

Réponse. (1) **Une VM tombe** : ses conteneurs sont morts jusqu'à l'intervention humaine ; dans un cluster, le *controller manager* constate que les Pods manquent et le *scheduler* les replace sur un nœud sain. (2) **Montée en charge** : passer de deux à six instances demande quatre VM à préparer ; dans un cluster, `replicas: 6` et le scheduler répartit. (3) **Mise à jour** : `compose up` recrée le conteneur, le service coupe quelques secondes sur chaque VM ; le Deployment fait un *rolling update* (labo 12), nouveau Pod prêt avant de retirer l'ancien, piloté par le controller manager. S'y ajoute l'auto-réparation sur **panne de la machine**, que `restart: always` ne couvre pas.

Pourquoi. Compose agit sur une machine, à la demande ; le control plane agit sur toutes, en permanence, à partir d'un état voulu.

Nuance. Pour une application interne sans exigence de disponibilité, les deux VM Compose sont une réponse raisonnable (labo 09). La différence se paie en complexité.

Exemple.

```
kubectl get deployment api -o jsonpath='{.spec.replicas} voulus, {.status.readyReplicas} prêts'
```

Question 2 — `apply`, `apply`, `create`

Réponse. `pod/api created`, puis `pod/api unchanged`, puis `Error from server (AlreadyExists): … pods "api" already exists`. **Déclaratif** : le fichier décrit un état, pas une action ; `apply` compare l'état voulu à l'objet existant et n'agit que sur la différence. **Idempotent** : appliquer N fois le même fichier produit le même résultat qu'une fois — ce qui rend `apply` sûr en CI, dans un script, ou après une panne. `create` est impératif : « crée », donc échoue si l'objet existe.

Pourquoi. `apply` enregistre la dernière configuration appliquée (annotation `kubectl.kubernetes.io/last-applied-configuration`) et calcule un *patch* à trois voies.

Nuance. Certains champs sont **immuables** sur un Pod (l'image, par exemple) : un `apply` qui les change échoue, il faut supprimer et recréer. Les Deployments (labo 12) existent précisément pour changer l'image sans cette gêne.

Exemple.

```
kubectl apply -f pod-api.yaml ; kubectl apply -f pod-api.yaml      # created, puis unchanged
kubectl get pod api -o jsonpath='{.metadata.annotations}' | cut -c1-80
```

Question 3 — Un Pod seul, un Pod sous Deployment

Réponse. Le dernier `get pods` montre **un seul** Pod, `api-xxxxxxxxx-yyyyy`, créé il y a quelques secondes ; `web` a disparu. `web` est un Pod **sans propriétaire** : supprimé, personne ne réclame son retour. Le Pod de `api` appartient à un **ReplicaSet**, lui-même créé par le **Deployment** : la

boucle de réconciliation du ReplicaSet constate `0 < 1` réplica et en crée un nouveau, avec un autre suffixe.

Pourquoi. Chaque objet peut porter une `ownerReference` ; un contrôleur surveille les objets qu'il possède et maintient le compte.

Nuance. Le nouveau Pod a une **nouvelle IP** et un nouveau nom : rien de ce qui pointait vers l'ancien ne doit avoir été codé en dur — argument des Services du labo 13.

Exemple.

```
kubectl get pod -l app=api -o jsonpath='{.items[0].metadata.ownerReferences[0].kind}'   # ReplicaSet
```

Question 4 — `ImagePullBackOff` sur une image locale

Réponse. Le kubelet a demandé à **son** containerd de tirer `localhost/api:1.0` ; ce containerd ne connaît pas l'image (elle est dans le stockage de Podman, un autre répertoire, un autre utilisateur) et tente un pull depuis un registry `localhost` qui n'existe pas : `ErrImagePull`, puis `ImagePullBackOff` (nouvelles tentatives espacées). Solution : `podman save localhost/api:1.0 | sudo k3s ctr images import -` ; le kubelet réessaie et trouve l'image. Le champ qui compte : `imagePullPolicy: IfNotPresent` (ou `Never`), sans quoi `Always` ferait échouer même une image importée.

Pourquoi. Deux runtimes, deux magasins : Podman rootless (`~/.local/share/containers`) et containerd de k3s (`/var/lib/rancher/k3s`).

Nuance. `Always` est le défaut quand le tag est `latest` ou absent ; `IfNotPresent` pour tout autre tag. Écrire la politique explicitement évite la surprise.

Exemple.

```
sudo k3s crictl images | grep api        # vide avant l'import, une ligne après
kubectl describe pod api | grep -A2 'Failed to pull'
```

Question 5 — Le Pod invisible

Réponse. Le Pod est dans un **autre namespace** que celui du contexte courant (`default`). Pour le voir : `kubectl get pods -n lab11`, ou `-A` (tous les namespaces). Pour ne plus répéter `-n` : `kubectl config set-context --current --namespace=lab11`, qui modifie le namespace par défaut du contexte dans le kubeconfig.

Pourquoi. Un namespace est une cloison : chaque commande `kubectl` s'applique à un seul, celui de l'option `-n` ou du contexte.

Nuance. Certains objets ne sont **pas** namespacés (nœuds, namespaces, StorageClasses, PersistentVolumes) : `kubectl api-resources` l'indique dans la colonne `NAMESPACED`.

Exemple.

```
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}'   # lab11
```

Question 6 — Un Pod ou deux

Réponse. (a) nginx écrit `proxy_pass http://localhost:8080/` : même network namespace, comme le pod Podman du **labo 07**. (b) nginx écrit `proxy_pass http://api:8080/`, nom résolu par un **Service** : comme le réseau user-defined et Compose du **labo 09**. En production, (b) : chaque service est mis à l'échelle, mis à jour et redémarré **indépendamment** ; avec (a), trois réplicas du front imposeraient trois réplicas de l'API, et une mise à jour de l'un redémarrerait l'autre.

Pourquoi. Le Pod est l'unité de placement et de réplication : ce qui est dans le même Pod vit et meurt ensemble.

Nuance. Le multi-conteneurs sert aux *sidecars* (agent de logs, proxy TLS) et aux *init containers* : des compagnons du conteneur principal, pas d'autres services.

Exemple.

```
kubectl explain pod.spec.initContainers | head -5
```

Question 7 — Le kubeconfig de k3s

Réponse. Trois éléments : le **cluster** (`server: https://127.0.0.1:6443` et le certificat de l'autorité), l'**utilisateur** (un certificat client et sa clé, qui est l'administrateur du cluster), et le **contexte** `default` qui les relie. En mode `644`, tout utilisateur de la machine lit la clé et devient administrateur : acceptable sur votre poste où vous êtes seul, inacceptable sur un serveur partagé. Copié ailleurs, le fichier pointe vers `127.0.0.1` : il faut remplacer l'adresse par celle de la machine k3s (et que le port 6443 soit joignable) — le certificat serveur accepte par défaut l'IP du nœud.

Pourquoi. Le kubeconfig est à la fois l'annuaire et le trousseau de clés de `kubectl`.

Nuance. En entreprise, le kubeconfig contient un *token* ou une authentification OIDC à durée limitée, pas un certificat administrateur éternel.

Exemple.

```
grep -E 'server:|client-certificate-data|name:' /etc/rancher/k3s/k3s.yaml | cut -c1-60
```

Question 8 — `localhost:8080 refused`

Réponse. `kubectl` ne trouve **aucun kubeconfig** et retombe sur son défaut historique, `http://localhost:8080`, où rien n'écoute (l'API server est en HTTPS sur 6443). Cela arrive quand on utilise un `kubectl` installé séparément (`apt`, `snap`), qui ne connaît pas `/etc/rancher/k3s/k3s.yaml`. Corrections : `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (dans `~/.bashrc`), ou `mkdir -p ~/.kube && cp /etc/rancher/k3s/k3s.yaml ~/.kube/config`.

Pourquoi. Le `kubectl` fourni par k3s est un lien vers le binaire `k3s`, qui sait où est son kubeconfig ; un `kubectl` générique ne le sait pas.

Nuance. Copier le fichier dans `~/.kube/config` en fait **votre** copie : si k3s régénère ses certificats, elle devient périmée. La variable `KUBECONFIG` suit toujours l'original.

Exemple.

```
which kubectl ; ls -l $(which kubectl)       # /usr/local/bin/kubectl -> k3s
KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get nodes
```

Question 9 — API server arrêté

Réponse. Les Pods **continuent de tourner** : le kubelet les a lancés et les surveille localement ; le trafic qui y arrive est servi. Vous ne pouvez plus rien **changer ni consulter** : `kubectl` échoue, aucun nouveau Pod n'est planifié, aucun Deployment n'est réconcilié. Un Pod qui meurt est relancé par le kubelet **sur le même nœud** (c'est lui qui applique la `restartPolicy`), mais s'il faut un autre nœud, personne ne décide. L'API server est donc la porte pour **modifier et observer** ; le moteur réel est le kubelet, qui garde en mémoire ce qu'il doit faire tourner.

Pourquoi. Kubernetes est conçu pour que le plan de données survive au plan de contrôle : chaque nœud est autonome pour ce qu'il a déjà.

Nuance. Avec k3s mono-nœud, « l'API server s'arrête » signifie `systemctl stop k3s`, qui arrête **aussi** le kubelet et containerd : les Pods meurent. L'autonomie des nœuds ne se voit que sur un vrai cluster.

Exemple.

```
kubectl cluster-info          # Kubernetes control plane is running at https://127.0.0.1:6443
kubectl get --raw /healthz    # ok
```

Question 10 — Les Events d'un démarrage

Réponse. `Scheduled` — le **scheduler** a choisi le nœud `wsl-ubuntu` (ici le seul). `Pulling` / `Pulled` — le **kubelet** a demandé à containerd de tirer l'image (3,75 s). `Created` / `Started` — le kubelet a créé puis démarré le conteneur via containerd. Face à `podman run` du labo 03 : le pull, la création et le démarrage sont les mêmes étapes, faites par `podman` et `crun` ; le cluster ajoute la **planification** (choisir une machine), l'**enregistrement** de chaque étape comme Event dans l'API server, et la **surveillance continue**. Avec un seul nœud, le scheduler n'a aucun choix : il « travaille » sans décider.

Pourquoi. Chaque composant publie des Events sur l'objet qu'il traite ; `describe` les rassemble dans l'ordre.

Nuance. Les Events sont conservés une heure par défaut : pour un incident ancien, il faut les logs de l'application ou une collecte externe.

Exemple.

```
kubectl get events --sort-by=.lastTimestamp | tail -5
```

Question 11 — `Pending` sans IP

Réponse. Soit **aucun nœud ne convient** (ressources insuffisantes, nœud `NotReady`, contrainte de placement), soit **l'image ou le volume bloque** avant l'attribution — mais sans IP du tout, c'est presque toujours le scheduler. `kubectl describe pod` tranche : `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory` d'un côté ; `Scheduled` suivi d'un `Failed to pull image` ou d'un `FailedMount` de l'autre (avec une IP déjà attribuée).

Pourquoi. L'IP est attribuée par le CNI au moment où le kubelet crée le Pod sur un nœud ; pas de nœud, pas d'IP.

Nuance. Un Pod qui demande `resources.requests.memory: 8Gi` sur un WSL à 4 Go restera `Pending` pour toujours, sans message d'erreur ailleurs que dans ses Events.

Exemple.

```
kubectl describe pod api | grep -A3 '^Events' ; kubectl get nodes ; kubectl describe node |
grep -A5 'Allocated resources'
```

Question 12 — De k3s à EKS

Réponse. À demander ou configurer : (1) un **Ingress controller** (Traefik est fourni par k3s ; sur EKS on installe ingress-nginx ou on utilise l'ALB) ; (2) une **StorageClass** par défaut (`local-path` chez k3s ; sur EKS, EBS via un pilote CSI) ; (3) les Services `LoadBalancer` (ServiceLB chez k3s ; un équilibreur cloud, payant, sur EKS) ; (4) l'**accès aux images** : plus de `k3s ctr images import`, les images viennent d'un registry (ECR) avec des droits de pull. Ce qui ne change pas : les **manifestes** (Pod, Deployment, Service, ConfigMap…) et les commandes `kubectl`.

Pourquoi. k3s empaquette des choix que les clusters managés laissent ouverts, parce qu'ils dépendent de l'infrastructure du client.

Nuance. Même sur EKS, l'équipe plateforme a souvent déjà fait ces choix : le développeur les **découvre** (`kubectl get storageclass`, `kubectl get ingressclass`) plutôt qu'il ne les installe.

Exemple.

```
kubectl get storageclass ; kubectl get ingressclass ; kubectl get svc -n kube-system traefik
```

Question 13 — `hostPath` et `hostPort`

Réponse. `hostPath` monte un fichier **de la machine qui exécute le Pod** : sur un cluster, le Pod peut atterrir sur n'importe quel nœud, où ce fichier n'existe pas — et un Pod qui lit le disque des nœuds est une faille (un `hostPath` sur `/etc` lit la configuration de l'hôte). `hostPort` réserve un port **du nœud** : deux réplicas sur le même nœud entrent en conflit, et le client doit connaître l'adresse du nœud. Remplacements : la configuration nginx devient une **ConfigMap** montée en volume (labo 14) ; l'exposition devient un **Service** puis un **Ingress** (labo 13), qui ne dépendent d'aucun nœud.

Pourquoi. Un manifeste de cluster doit être **indépendant de la machine** : tout ce dont le Pod a besoin est un objet de l'API, pas un chemin local.

Nuance. `hostPath` et `hostPort` ont des usages légitimes pour les **agents de nœud** (collecteur de logs, supervision) déployés en DaemonSet, un par machine.

Exemple.

```
grep -n -E 'hostPath|hostPort' ~/labo-docker/11/demo-pod.yaml
```